# Technical Transition Letter — Exploro OS

**Project:** Exploro OS ( `exploro-app` ) **Prepared by:** Dhiren Kirpalani **Date:** October 12, 2026 **Repository:** `https://github.com/PROYECT888/Private-GPT.git` **Production domain:** `https://exploro-os.com` (and `https://www.exploro-os.com` ) **Handover deadline:** October 15, 2026

---

## 1. Executive Summary

Exploro OS is a private, AI-powered business platform for SMEs. Users train an AI on their own documents (knowledge base), then deploy it across channels their team and customers already use — Email (Gmail, Outlook, Zoho, iCloud, Hostinger, GoDaddy), WhatsApp Business, Telegram, Slack, Google Calendar, Google Drive, Google Meet, and Calendly.

The app is a **Next.js 14 (App Router)** monorepo written in **TypeScript**, using **Supabase** (Postgres + Auth + Storage + Realtime) as the backend, **DeepSeek** as the LLM, **Stripe** (US) and **Polar.sh** (non-US) for payments, and a self-hosted **Evolution API** on a Hostinger VPS for WhatsApp QR-based pairing.

This document covers architecture, codebase status, repositories, database, APIs, integrations, hosting, known issues, technical debt, and operational runbooks so a new developer can take over immediately.

---

## 2. Tech Stack

| Layer | Technology |
|---|---|
| Framework | Next.js 14.2 (App Router) |
| Language | TypeScript 5.3 |
| Styling | Tailwind CSS 3.4 + `tailwindcss-animate` |
| UI Components | shadcn/ui + Radix UI |
| Icons | Lucide React + React Icons |
| Backend / Auth / DB | Supabase (Postgres, Auth, Storage, Realtime, RLS) |
| AI | DeepSeek API ( `deepseek-v4-flash` model, streaming SSE) |
| Web Search | Serper API ( `google.serper.dev` ) |
| Transactional Email | Resend |
| Payments (US) | Stripe (seat-based billing for Team plan) |
| Payments (non-US) | Polar.sh (Merchant of Record, API pinned to `2026-04` ) |
| Document Parsing | `pdf-parse` , `pdfjs-dist` , `mammoth` , `xlsx` , `pptx-parser` /jszip |
| Markdown | `react-markdown` + `remark-gfm` |
| Telegram MTProto | `teleproto` (GramJS fork) |
| WhatsApp QR | Evolution API (self-hosted, Hostinger VPS) |
| State | React Context API ( `AuthContext` , `WorkspaceContext` ) |
| Fonts | Inter (Google Fonts) |

| | |
|---|---|
| Process Manager (VPS) | PM2 |
| Reverse Proxy (VPS) | Nginx + certbot (Let's Encrypt) |

---

## 3. Current Architecture & Technical Structure

### 3.1 High-Level Architecture




### 3.2 Project Structure

```
Private-GPT/
├── app/
│   ├── auth-provider.tsx          # AuthContext — user, session, role, subscription
│   ├── workspace-provider.tsx     # WorkspaceContext — multi-workspace scoping
│   ├── layout.tsx                 # Root layout
│   ├── page.tsx                   # Landing page (server component, SEO metadata)
│   ├── home-client.tsx            # Landing page client component
│   ├── login/                     # Login page
│   ├── signup/                    # Signup (password strength, invite-aware)
│   ├── forgot-password/           # Password reset request
│   ├── reset-password/            # Set new password
│   ├── chat/                      # AI chat workspace (~4000+ lines, monolith)
│   ├── share/[id]/                # Public read-only shared conversation
│   ├── knowledge/                 # Knowledge base (documents + categories)
│   ├── channels/                  # Integrations hub (connect OAuth providers)
│   ├── crm/                       # CRM (Overview/Email/Messages/Calendar tabs, ~4600+
lines)
│   ├── contacts/                  # STUB — "Coming Soon" (functionality in /crm)
│   ├── inbox/                     # STUB — "Coming Soon" (functionality in /crm + /chat)
│   ├── agents/                    # STUB — "Coming Soon"
│   ├── analytics/                # STUB — "Coming Soon"
│   ├── automations/              # STUB — "Coming Soon"
│   ├── workspace/[id]/           # Workspace settings + member management
│   ├── pricing/                  # Pricing (Solo/Team/Enterprise)
│   ├── admin/                    # Super admin panel (~2000+ lines)
│   ├── transify/                 # Super-admin translation manager (ES overrides)
│   ├── support/                  # Support ticket form
│   ├── profile/                  # User + company + AI + branding settings
│   ├── invite/                   # Workspace invitation accept/decline
│   ├── creating/                 # Post-onboarding cosmetic loading screen
│   ├── construction/             # "Coming Soon" email-capture (localStorage only)
│   ├── about/                    # Static marketing page
│   ├── privacy/, terms/, disclaimer/
│   └── api/                      # 96 API routes (see §6)
├── components/
│   ├── nav-rail.tsx              # Vertical side nav
│   ├── navbar.tsx                # Top navigation bar
│   ├── workspace-selector.tsx    # Workspace switcher dropdown
│   ├── create-workspace-modal.tsx
│   ├── notification-bell.tsx     # In-app notifications
│   ├── password-strength.tsx    # Animated password strength bar
│   ├── trial-pill.tsx           # Trial countdown pill
│   ├── trial-paywall.tsx        # Full-screen paywall on trial expiry
│   ├── announcement-banner.tsx  # Admin-controlled global banner
│   ├── checkout-button.tsx      # Stripe/Polar routing
│   ├── google-picker.tsx        # Google Drive file picker
│   ├── email-edit-form.tsx      # Inline email compose/edit
│   ├── cinematic-background.tsx # Animated gradient background
│   ├── animated-placeholder.tsx # Rotating chat input suggestions
│   ├── footer.tsx, faq-section.tsx
│   └── ui/                      # shadcn/ui primitives (button, input, toast, skeleton…)
├── lib/
│   ├── supabase.ts              # Supabase client + all DB/Storage helpers (~2000 lines)
```

```
│   ├── subscription.ts          # getSubscription, startTrial, isTrialExpired
│   ├── workspace.ts             # Workspace CRUD + member helpers
│   ├── polar.ts                 # Polar.sh API client (pinned to 2026-04)
│   ├── stripe.ts                # Stripe client init
│   ├── stripe-seats.ts          # Seat-based billing sync
│   ├── system-email.ts          # Transactional email (Resend)
│   ├── telegram.ts              # Telegram bot helpers
│   ├── telegram-user.ts         # Telegram MTProto user-session helpers
│   ├── token-limits.ts          # Per-plan token/message caps + quota checking
│   ├── app-settings.ts          # app_settings read/write
│   ├── theme-engine.ts          # Brand color compilation + dark mode derivation
│   ├── i18n.tsx                 # Internationalization context + hook
│   ├── translations.ts          # Full EN/ES translation strings (~124KB)
│   ├── file-types.ts            # Accepted file type definitions
│   ├── email-provider.ts        # Email provider detection + webmail URL helper
│   ├── api-logger.ts            # API request logger
│   ├── with-api-logging.ts      # HOF wrapper to log API route calls
│   ├── workspace-icons.ts       # Workspace icon helpers
│   └── utils.ts                 # cn() and utility helpers
├── supabase/
│   ├── schema.sql               # Base schema (profiles, translations)
│   └── migrations/              # 57 incremental SQL migrations (002–043 +
whatsapp_evolution)
├── sql/                          # trial-reminder cron setup SQL
├── scripts/                     # seed-translations.ts
├── tests/                       # web-search integration test
├── types/                       # TypeScript declarations (imap-simple, mailparser)
├── docs/                        # Internal documentation (this file + troubleshooting)
├── public/assets/images/        # exploro-logo.png, exploro-icon.svg
├── middleware.ts                # Route protection (auth cookie check)
├── next.config.mjs              # Next.js config (image remote patterns)
├── tailwind.config.ts
├── postcss.config.mjs
├── tsconfig.json
├── package.json                 # "exploro-app" v0.1.0
└── .env / .env.example          # Environment variables (see §8)
```

### 3.3 Authentication & Session Flow

1. **Supabase Auth** (email/password) — session stored in cookie `sb-auth-token` (max-age 7 days, `SameSite=Lax`). Cookie storage is configured in `lib/supabase.ts` so `middleware.ts` can read it server-side.
2. **`middleware.ts`** — public routes (`/`, `/login`, `/signup`, `/forgot-password`, `/reset-password`, `/pricing`, `/about`, `/disclaimer`, `/privacy`, `/terms`, `/support`, `/api/*`) bypass auth. All other routes require a non-empty `sb-auth-token` cookie, else redirect to `/login`.
3. **`AuthContext`** (`app/auth-provider.tsx`) — fetches user, session, profile, role, subscription once at root level. Caches role in `localStorage` (`exploro_role`) for instant restore. Auto-creates a 15-day trial subscription if a user has no `subscriptions` row.
4. **`WorkspaceContext`** (`app/workspace-provider.tsx`) — auto-creates "Admin's Workspace" for first-time users, backfills legacy NULL `workspace_id` records, restores last-selected workspace from

`localStorage`.

### 3.4 Multi-Workspace Model

- Each user belongs to one or more workspaces (owner or member).
- `workspaces` table (owner_id, name, description, icon).
- `workspace_members` table (workspace_id, user_id, role: `owner` / `admin` / `manager` / `member`).
- `workspace_invitations` table (token-based, email-scoped).
- Knowledge base (`documents`, `knowledge_categories`) and `chat_conversations` have optional `workspace_id` for scoping. **Note:** most other tables (email, calendar, whatsapp, telegram, slack, contacts) are **not** workspace-scoped — they are per-user (`user_id`).
- Seat billing: Team plan syncs Stripe subscription quantity to member count via `syncStripeSeats()`.

### 3.5 Subscription & Trial System

1. **Signup** → `startTrial()` creates a `subscriptions` row (`status: "trialing"`, `plan: "solo"`, `current_period_end = now + trial_days`).
2. **Trial active** → `TrialPill` shows countdown (gold `#FFBF00`).
3. **Trial expired** → `TrialPill` becomes "Subscribe"; `TrialPaywall` overlay on Chat/Knowledge/Channels/CRM (super_admin can dismiss).
4. **Subscribed** → webhook updates `status: "active"`.
5. **Payments**: US → Stripe (`/api/stripe/*`); non-US → Polar.sh (`/api/polar/*`). `CheckoutButton` routes based on `profile.location`.

### 3.6 AI Chat Pipeline

1. User sends message → `POST /api/chat` (streaming SSE).
2. **Usage guard**: `checkUsage(userId, sub)` checks token + message limits vs plan. Returns 429 if exceeded. Graceful degradation at >80% token quota (halves `max_tokens`).
3. **System prompt** assembled client-side + via `POST /api/ai/context` — injects:
   - Knowledge base document text (parsed via `/api/parse-document`).
   - Last 10 received emails.
   - Next 14 days of calendar events.
   - Last 10 WhatsApp/Slack/Telegram messages.
   - Calendly scheduling URL.
4. **DeepSeek** streaming response piped back via SSE. Usage stats logged (prompt cache hit ratio).
5. **Stream persistence**: module-level `_globalStream` singleton in `app/chat/page.tsx` keeps the SSE stream alive across page navigation. An 80ms `setInterval` polls it to render streaming text. On remount, full history is reloaded + partial streaming content restored.
6. **AI Actions**: AI embeds base64-encoded action blocks (`<!--ACTION_B64:...-->`) in responses. The client parses these and executes: `create_event`, `send_email`, `create_meet`, `send_whatsapp`, `search_contacts`, etc. Multi-step actions chain automatically (e.g., create calendar event → inject Meet link → send email).
7. **Long-term memory**: `/api/ai/memory/extract` runs every 6 messages, uses DeepSeek JSON mode to extract preferences/contacts/facts/decisions/instructions into `ai_long_term_memories` table.
8. **Web search**: AI can trigger `/api/web-search` → DeepSeek query rewrite → Serper multi-search → domain-authority rerank → context compression.

### 3.7 RAG / Knowledge Base Pipeline

1. **Upload** → drag-and-drop in `/knowledge` → file to `knowledge-base` Supabase Storage bucket (path: `${userId}/${filename}`) → `documents` row (`status: "INDEXED"`).
2. **Page count** → `/api/count-pages` (pdf-lib or mammoth word-count estimate).
3. **Text extraction** → `/api/parse-document` downloads from storage, extracts text (PDF: `pdf-parse` / `pdfjs-dist`, DOCX: `mammoth`, XLSX: `xlsx`, PPTX: jszip slide XML regex, TXT/MD/HTML/JSON/CSV: raw). Truncates to 12k chars. Stores parsed text in `documents.parsed_text`.
4. **Context injection** → `/api/ai/context` assembles KB content + emails + calendar + messages into the AI system prompt.
5. **Document lifecycle**: 30-day `expires_at` by default. `lazyArchiveDocuments` deletes storage files for expired non-pinned docs (sets `file_archived`). Pinning extends expiry. `extendDocumentExpiry` available.
6. **Google Drive import**: `useGooglePicker` → client-side download (incl. Google Workspace export) → base64 → `/api/drive/import` → storage + `documents` row → fires `/api/parse-document`. 12MB cap.

### 3.8 Dynamic Brand Theme Engine

`lib/theme-engine.ts` — `compileTheme(primary, secondary, style, mood)` generates a full `CompiledTheme`. Uses WCAG luminance math (`getLuminance`, `isDark`) + color helpers (`lighten`, `darken`, `toRgba`, `generateAccent`). Theme persisted to `localStorage` and synced from Supabase profile on load. Styles: cinematic, minimal, gradient, aurora. Moods: futuristic, calm, corporate, vibrant.

---

## 4. Current Codebase Status & Development Environment

### 4.1 Branch Strategy

- **`development`** (current active branch) — all recent work.
- **`main`** — production branch.
- **`backup-development`** — backup.
- **Remotes:**
  - `origin` → `https://github.com/PROYECT888/Private-GPT.git` (company repo)

### 4.2 Local Development Setup

```
# 1. Clone
git clone https://github.com/PROYECT888/Private-GPT.git
cd Private-GPT
git checkout development

# 2. Install dependencies
npm install

# 3. Configure environment
cp .env.example .env
# Fill in all variables (see §8 for the full list)

# 4. Apply database migrations
```

```
# Apply supabase/schema.sql first, then supabase/migrations/*.sql in order
# Either via `supabase db push` (CLI) or manually in Supabase SQL Editor

# 5. Run dev server
npm run dev
# Open http://localhost:3000
```

### 4.3 Build & Scripts

| Command | Description |
|---|---|
| `npm run dev` | Start dev server |
| `npm run build` | Production build |
| `npm start` | Start production server |
| `npm run lint` | ESLint |
| `npm run test:web-search` | Web search integration test ( `npx tsx tests/web-search.test.ts` ) |

### 4.4 Codebase Stats

- **96 API routes** across 15 domains.
- **~20 database tables**.
- **57 SQL migrations**.
- **Two monolith pages**: `app/chat/page.tsx` (~~4000+ lines) and~~ ~~`app/crm/page.tsx`~~ ~~(~~4600+ lines) — prime refactor candidates.
- **`lib/supabase.ts`** is ~2000 lines — single shared client + all DB helpers.
- **i18n**: EN/ES with ~124KB of translation strings in `lib/translations.ts` + DB overrides via `translations` table.

---

## 5. Database Structure, Schemas & Dependencies

### 5.1 Supabase Project

- **URL**: `NEXT_PUBLIC_SUPABASE_URL` (see `.env` )
- **Two keys**: anon key (client-side, RLS-enforced) + service role key (server-side, bypasses RLS).
- **Region**: configured in Supabase dashboard.

### 5.2 Tables

| Table | Purpose | Key Columns |
|---|---|---|
| `profiles` | User + company + AI config | `user_id` (FK auth.users, unique), `full_name` , `company_name` , `ai_` ( `user` / `manager` / `admin` / `super_admin` ), `avatar_url` , `logo_url` `trial_days` , `email_keywords` |
| `translations` | DB-driven i18n overrides | `key` , `lang` , `value` (unique `key,lang` ) |
| `documents` | Knowledge base files | `id` , `user_id` , `workspace_id` , `category` , `filename` , `original` `file_size_bytes` , `status` ( `INDEXED` / `PROCESSING` / `ERROR` ), `pa` `pinned` , `file_archived` , `parsed_text` |

| | | |
|---|---|---|
| knowledge_categories | Per-user document categories | id , user_id , name , sort_order , workspace_id |
| email_connections | Email provider credentials | user_id , provider (unique user_id,provider ), email_addres imap_host/port , status , last_error , cc_address , last_fe |
| email_messages | Inbox/sent emails | id , user_id , connection_id , provider , direction ( sent / subject , body , html_body , message_id , thread_id , read , |
| chat_conversations | AI chat conversations | id , user_id , title , workspace_id |
| chat_messages | Chat messages | id , conversation_id , role , content , sources , prompt_tok total_tokens , cache_hit_tokens |
| contacts | CRM contacts | id , user_id , name , email (unique user_id,email ), company starred , source ( email_import / whatsapp_import / telegram_import / slack_im last_contact , deal_value , deal_stage |
| calendar_connections | Calendar/Drive/Meet OAuth | id , user_id , provider ( google / calendly / googledrive / go scheduling_url , + OAuth tokens |
| calendar_events | Calendar events | id , user_id , connection_id , summary , description , start location , event_link , is_online |
| whatsapp_connections | Meta WhatsApp Cloud API | id , user_id , phone_number_id , access_token , phone_numbe webhook_verified |
| whatsapp_messages | WhatsApp messages | id , user_id , connection_id , session_id , direction , from body , media_url , media_type , timestamp , read |
| whatsapp_sessions | Evolution API sessions | id , user_id , instance_name , phone_number , status ( conne provider ( evolution / meta ) |
| telegram_connections | Telegram bot tokens | id , user_id , bot_token , bot_username , bot_first_name , s |
| telegram_messages | Telegram messages | id , user_id , connection_id , direction , chat_id , chat_ty from_first_name , from_last_name , from_username , from_ph read |
| telegram_user_sessions | MTProto user sessions | user_id , phone_number , session_string , tg_user_id , tg_u tg_last_name , status ( connected / expired / pending ), phon |
| slack_connections | Slack OAuth tokens | id , user_id , team_id , team_name , bot_user_id , bot_acces |
| slack_messages | Slack DM messages | id , user_id , connection_id , direction , channel_id , char slack_user_name , slack_ts (unique user_id,slack_ts ), body |
| subscriptions | Billing state | id , user_id , stripe_customer_id , stripe_subscription_id polar_subscription_id , polar_customer_id , status , plan current_period_start , current_period_end , cancel_at_per |
| workspaces | Workspaces | id , owner_id , name , description , icon |
| workspace_members | Workspace membership | id , workspace_id , user_id , role ( owner / admin / manager / |
| workspace_invitations | Pending invites | id , workspace_id , invited_email , token , status , expires |
| notifications | In-app notifications | id , user_id , type , title , body , data (json), read , create |
| crm_kanban_cols | Kanban column config | user_id , board ( email / messages / calendar ), col_id , labe |

| | | |
|---|---|---|
| `crm_kanban_card_cols` | Card-to-column mapping | `user_id` , `board` , `card_id` , `col_id` |
| `app_settings` | Global settings (KV) | `key` , `value` , `updated_at` — stores `trial_days` , announcement co |
| `admin_audit_log` | Admin action audit | `id` , `admin_user_id` , `action` , `target_user_id` , `details` , `cre` |
| `api_logs` | API request logs | route, method, status, duration, user_id, etc. |
| `api_stats_hourly` | Aggregated API stats | hourly aggregation with p50/p95/p99 |
| `ai_long_term_memories` | AI extracted memories | `user_id` , `type` , `content` , `active` , `last_accessed_at` |
| `notification_log` | Dedup log for cron emails | prevents duplicate trial-reminder emails |

### 5.3 Storage Buckets

| Bucket | Purpose | Public? |
|---|---|---|
| `avatars` | User avatar images | Yes (public URL) |
| `logos` | Company logos | Yes (public URL) |
| `knowledge-base` | Uploaded KB documents | Yes (public URL via `getDocumentPublicUrl` ) |
| `support-screenshots` | Support ticket screenshots | Yes |
| `chat-media` | Chat attachment uploads (auto-created) | Yes |
| `telegram-media` | Telegram media downloads (auto-created) | Yes |

### 5.4 RLS (Row Level Security)

- Enabled on `profiles` , `translations` , `documents` , `knowledge_categories` , `subscriptions` , and most user-scoped tables.
- Policies generally allow users to read/write only their own rows ( `auth.uid() = user_id` ).
- **Caveat**: Most API routes use the **service-role key** which bypasses RLS entirely. Security depends on the API layer filtering by `userId` — which is currently client-supplied (see §10 Security).
- `whatsapp_sessions` and `whatsapp_messages` writes use `createAdminClient()` (service-role) because Evolution API webhooks write without a user session.

### 5.5 Realtime

Supabase Realtime is enabled on key tables ( `017_enable_realtime.sql` ). The client subscribes via `postgres_changes` filtered by `user_id=eq.{userId}` for: email messages, calendar events, contacts, WhatsApp messages, Telegram messages, Slack messages, and notifications.

### 5.6 Triggers

- `on_auth_user_created` → `private.handle_new_user()` → auto-creates a `profiles` row on signup.
- `profiles_updated_at` → `handle_updated_at()` → auto-updates `updated_at` on profile changes.

### 5.7 Migrations

57 migration files in `supabase/migrations/`. Apply in order. Key milestones:

- `schema.sql` — base (profiles, translations)
- `004` — knowledge base tables
- `010–016` — email, chat, contacts, calendar, WhatsApp
- `017` — realtime
- `019` — subscriptions
- `025` — workspaces + Polar.sh (replaced Lemon Squeezy)
- `028–031` — API logging + stats
- `034` — token usage tracking
- `036–041` — Telegram media, email dedup/attachments
- `whatsapp_evolution.sql` — Evolution API sessions table

---

# 6. APIs, Integrations, Webhooks & Authentication

## 6.1 API Routes (96 total)

**Chat / AI**

| Route | Method | Description |
|---|---|---|
| `/api/chat` | POST | DeepSeek streaming proxy + usage guard |
| `/api/chat/title` | POST | Auto-generate conversation title |
| `/api/chat/cleanup` | POST | Strip ACTION blocks from messages |
| `/api/ai/context` | POST | RAG context assembly (KB + emails + calendar + messages) |
| `/api/ai/memory/extract` | POST | Extract long-term memories from conversations |
| `/api/ai/memory/fetch` | GET | Fetch top-50 memories |
| `/api/web-search` | POST | Serper web search with DeepSeek query rewrite |
| `/api/context` | POST | Fetch arbitrary URL → text (SSRF risk — see §10) |
| `/api/parse-document` | POST | Document text extraction |
| `/api/count-pages` | POST | PDF page count |
| `/api/usage` | POST | Token usage summary |
| `/api/upload-media` | POST | Upload to `chat-media` bucket |

**Email**

| Route | Method | Description |
|---|---|---|
| `/api/email/fetch` | POST | Import from Gmail/Outlook/IMAP (1245 lines, keyword filter) |
| `/api/email/send` | POST | Send via Gmail API / Microsoft Graph / SMTP |
| `/api/email/messages` | GET | List email_messages |
| `/api/email/disconnect` | POST | Delete connection + messages + contacts |

| | | |
|---|---|---|
| `/api/email/test` | POST | Verify SMTP credentials |
| `/api/email/welcome` | POST | Send welcome email (unauthenticated) |
| `/api/email/oauth/gmail/connect` | GET | Gmail OAuth redirect |
| `/api/email/oauth/outlook/connect` | GET | Outlook OAuth redirect |
| `/api/email/oauth/callback` | GET | Google/Microsoft OAuth callback |

**Calendar / Meet / Drive**

| Route | Method | Description |
|---|---|---|
| `/api/calendar/fetch` | POST | Fetch events from Google Calendar + Calendly |
| `/api/calendar/create` | POST | Create Google Calendar event (+ optional Meet link) |
| `/api/calendar/delete` | POST/DELETE | Delete Google event + DB row |
| `/api/calendar/oauth/google/connect` | GET | Google Calendar OAuth |
| `/api/calendar/oauth/callback` | GET | Google OAuth callback |
| `/api/calendly/oauth/connect` | GET | Calendly PKCE OAuth |
| `/api/calendly/oauth/callback` | GET | Calendly token exchange |
| `/api/calendly/fetch` | POST | Fetch Calendly events |
| `/api/meet/oauth/google/connect` | GET | Google Meet OAuth |
| `/api/meet/oauth/callback` | GET | Google Meet OAuth callback |
| `/api/drive/oauth/google/connect` | GET | Google Drive OAuth |
| `/api/drive/oauth/callback` | GET | Google Drive OAuth callback |
| `/api/drive/fetch` | POST | List Drive files |
| `/api/drive/folders/create` | POST | Create Drive folder |
| `/api/drive/upload` | POST | Upload to Drive |
| `/api/drive/token` | GET | **Returns raw Google access token** (sensitive) |
| `/api/drive/import` | POST | Import Drive file to KB |

**WhatsApp**

| Route | Method | Description |
|---|---|---|
| `/api/whatsapp/webhook` | GET/POST | Meta webhook (GET verify, POST store messages) |
| `/api/whatsapp/oauth/connect` | GET | Meta OAuth redirect |
| `/api/whatsapp/oauth/callback` | GET | Meta OAuth callback |
| `/api/whatsapp/embedded-signup` | POST | Server-side Embedded Signup |
| `/api/whatsapp/send` | POST | Send via Evolution or Meta Cloud API |
| `/api/whatsapp/status` | GET | Poll Evolution connection state |
| `/api/whatsapp/qr` | GET/POST/DELETE | Evolution QR create/refresh/delete |

| | | |
|---|---|---|
| `/api/whatsapp/qr-refresh` | POST | Re-fetch QR |
| `/api/whatsapp/evolution/sync` | POST | Bulk sync messages + contacts from VPS |
| `/api/whatsapp/evolution/webhook` | POST | Evolution event webhook |
| `/api/whatsapp/evolution/send` | POST | Send via Evolution |
| `/api/whatsapp/evolution/disconnect` | POST | Disconnect Evolution instance |

**Telegram**

| Route | Method | Description |
|---|---|---|
| `/api/telegram/user/send-code` | POST | Send login code |
| `/api/telegram/user/verify-code` | POST | Verify code + 2FA password |
| `/api/telegram/user/disconnect` | POST | Delete session + messages + contacts |
| `/api/telegram/user/fetch-chats` | POST | Fetch dialogs + messages (MTProto) |
| `/api/telegram/user/fetch-media` | POST | Download photo media |
| `/api/telegram/user/fetch-contacts` | POST | Fetch contacts |
| `/api/telegram/user/send` | POST | Send message or file |
| `/api/telegram/user/upload-media` | POST | Upload to `telegram-media` bucket |

**Slack**

| Route | Method | Description |
|---|---|---|
| `/api/slack/oauth/callback` | GET | Slack OAuth callback |
| `/api/slack/events` | POST | Slack Events API webhook (**signature verification is a stub — see §10**) |
| `/api/slack/fetch` | POST | Import DM history |
| `/api/slack/send` | POST | Send message |
| `/api/slack/disconnect` | POST | Disconnect + revoke |

**Payments**

| Route | Method | Description |
|---|---|---|
| `/api/stripe/checkout` | POST | Create checkout session |
| `/api/stripe/portal` | POST | Billing portal |
| `/api/stripe/webhook` | POST | Stripe webhook (signature verified) |
| `/api/polar/checkout` | POST | Polar checkout |
| `/api/polar/webhook` | POST | Polar webhook (signature verified if secret set) |
| `/api/subscriptions/start-trial` | POST | Start 15-day trial |

**Admin (super_admin gated)**

| Route | Method | Description |
|---|---|---|

| | | |
|---|---|---|
| `/api/admin/stats` | GET | Dashboard metrics (MRR, churn, etc.) |
| `/api/admin/api-monitoring` | GET | API stats with percentiles |
| `/api/admin/audit-log` | GET | Audit log |
| `/api/admin/companies` | GET | Companies + workspaces + channels |
| `/api/admin/custom-trials` | GET | Per-user trial overrides |
| `/api/admin/lookup-user` | GET | Email → user lookup |
| `/api/admin/notifications` | GET | Notification log |
| `/api/admin/reset-usage` | POST | Reset user usage period |
| `/api/admin/set-role` | POST | Set user role |
| `/api/admin/settings` | POST | Update app_settings |
| `/api/admin/update-limits` | POST | Update token/message limits |
| `/api/admin/update-trial` | POST | Update global trial days |
| `/api/admin/user-trial` | POST | Per-user trial override |
| `/api/admin/user-usage` | GET | Per-user usage |

**Workspace**

| Route | Method | Description |
|---|---|---|
| `/api/workspace/invite` | POST | Send invitation |
| `/api/workspace/invitations` | GET | List pending invitations |
| `/api/workspace/invitation` | GET | Get invitation by token |
| `/api/workspace/accept-invite` | POST | Accept invitation |
| `/api/workspace/decline-invite` | POST | Decline invitation |
| `/api/workspace/remove-member` | POST | Remove member |

**Misc**

| Route | Method | Description |
|---|---|---|
| `/api/app-settings` | GET | Public app settings (announcement + limits) |
| `/api/meta/config` | GET | Meta App ID for FB SDK |
| `/api/support` | POST | Support ticket email (Resend) |
| `/api/messages/mark-read` | POST | Mark message read (WhatsApp/Telegram/Slack) |
| `/api/cron/trial-reminder` | GET | Cron job (CRON_SECRET bearer) — trial expiry emails |

## 6.2 OAuth Flows

- **Google** (Gmail, Calendar, Drive, Meet): separate redirect URIs ( `/api/email/oauth/callback` , `/api/calendar/oauth/callback` , `/api/drive/oauth/callback` , `/api/meet/oauth/callback` ). `state` = base64 `{userId, provider}` — **not signed**.

- **Microsoft** (Outlook): `/api/email/oauth/callback` (shared with Google).
- **Meta** (WhatsApp): `/api/whatsapp/oauth/callback`, popup mode with `postMessage`.
- **Slack**: `/api/slack/oauth/callback`, `state` = raw userId.
- **Calendly**: PKCE (S256), code_verifier in `state` (insecure but functional).

### 6.3 API Logging

`lib/with-api-logging.ts` — HOF wrapper logs route, method, duration, status to `api_logs` table. Aggregated into `api_stats_hourly` with p50/p95/p99 percentiles. Not all routes are wrapped (see §10).

---

## 7. Third-Party Services & Integrations

| Service | Purpose | Env Vars | Notes |
|---|---|---|---|
| **DeepSeek** | LLM (chat, title gen, memory extraction, web search query rewrite) | `DEEPSEEK_API_KEY`, `DEEPSEEK_URL`, `DEEPSEEK_MODEL` | Model: `deepseek-v4-flash`. Streaming SSE. Prompt cache hit tracking. |
| **Supabase** | Auth, Postgres, Storage, Realtime | `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`, `NEXT_PUBLIC_SUPABASE_SERVICE_ROLE_KEY` | Service role key bypasses RLS |
| **Serper** | Web search | `SERPER_API_KEY`, `SERPER_URL` | `google.serper.dev/search` |
| **Resend** | Transactional email | `RESEND_API_KEY` | From: `no-reply@exploro-os.com` |
| **Stripe** | US payments | `NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY`, `STRIPE_APIKEY_TOKEN`, `STRIPE_WEBHOOK_SECRET`, `STRIPE_PRICE_SOLO`, `STRIPE_PRICE_TEAM` | Seat-based billing for Team plan |
| **Polar.sh** | Non-US payments (MoR) | `POLAR_ACCESS_TOKEN`, `POLAR_WEBHOOK_SECRET`, `POLAR_PRODUCT_SOLO`, `POLAR_PRODUCT_TEAM` | API pinned to `2026-04` — update quarterly |
| **Google OAuth** | Gmail, Calendar, Drive, Meet | `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET` | 4 redirect URIs |
| **Microsoft OAuth** | Outlook | `MICROSOFT_CLIENT_ID`, `MICROSOFT_CLIENT_SECRET` | |
| **Meta** | WhatsApp, Instagram, Facebook | `META_APP_ID`, `META_APP_SECRET`, `META_WHATSAPP_CONFIG_ID`, `WHATSAPP_WEBHOOK_VERIFY_TOKEN` | |
| **Slack** | Slack integration | `NEXT_PUBLIC_SLACK_CLIENT_ID`, `SLACK_CLIENT_ID`, `SLACK_CLIENT_SECRET`, `SLACK_BOT_TOKEN` | |
| **Telegram** | User account (MTProto) | `TELEGRAM_API_ID`, `TELEGRAM_API_HASH` | Uses `teleproto` library |
| **Calendly** | Scheduling | `CALENDLY_CLIENT_ID`, `CALENDLY_CLIENT_SECRET`, | PKCE OAuth |

| | | | |
|---|---|---|---|
| | | `CALENDLY_WEBHOOK_SIGNING_KEY` | |
| **Evolution API** | WhatsApp QR pairing | `NEXT_PUBLIC_WHATSAPP_PROVIDER`, `EVOLUTION_API_URL`, `EVOLUTION_API_KEY` | Self-hosted on Hostinger VPS |

---

## 8. Hosting, Cloud, Server, Domain & Infrastructure

### 8.1 Application Hosting

- **Platform**: Vercel (Next.js native). The app is a Next.js 14 App Router project.
- **Production domain**: `https://exploro-os.com` and `https://www.exploro-os.com`.

### 8.2 Database & Auth

- **Supabase** — hosted Postgres + Auth + Storage + Realtime.
- Project URL and keys in `.env` (transfer to new team — see §11).

### 8.3 WhatsApp VPS (Evolution API)

- **VPS**: Hostinger KVM — `srv1912272.hstgr.cloud` (Ubuntu 22.04+)
- **Stack**: Node.js 20 + PM2 + PostgreSQL + Redis + Nginx + certbot (Let's Encrypt)
- **Evolution API**: `/opt/evolution-api/` (git clone of `Atendai/evolution-api`)
- **Config**: `/opt/evolution-api/.env` — `AUTHENTICATION_API_KEY`, `DATABASE_URL`, `WEBHOOK_GLOBAL_URL`
- **Current `EVOLUTION_API_URL`**: `http://srv1912272.hstgr.cloud` (HTTP, not HTTPS — no domain pointed yet)
- **Full setup guide**: `VPS_SETUP_GUIDE.md` in the repo root.
- **PM2 commands**: `pm2 status`, `pm2 logs evolution-api`, `pm2 restart evolution-api`
- **Firewall**: UFW — ports 22 (SSH), 80 (HTTP), 443 (HTTPS) only.
- **PostgreSQL + Redis**: localhost only.

### 8.4 Domains

- `exploro-os.com` — primary app domain
- `www.exploro-os.com` — www variant

### 8.5 Email Infrastructure

- **Transactional**: Resend (`no-reply@exploro-os.com`)
- **Support inbox**: `support@exploro-os.com`

---

## 9. Known Technical Limitations & Architectural Constraints

### 9.1 Architecture Constraints

1. **Client-supplied userId**: All API routes trust a `userId` from the request body/query with no server-side session verification. This is an IDOR vulnerability — see §10.
2. **Service-role key in client bundle**: `NEXT_PUBLIC_SUPABASE_SERVICE_ROLE_KEY` is `NEXT_PUBLIC_` prefixed, meaning it may be exposed to the browser. `createAdminClient()` in

`lib/supabase.ts` uses it. All Evolution API functions use the admin client, bypassing RLS.

3. **No vector embeddings**: RAG is currently keyword/substring-based. The roadmap includes pgvector + embeddings but this is not implemented. Context injection pulls full document text (truncated to 12k chars), not semantic chunks.
4. **Workspace scoping incomplete**: Only `documents`, `knowledge_categories`, and `chat_conversations` have `workspace_id`. Email, calendar, WhatsApp, Telegram, Slack, and contacts are per-user only, not per-workspace.
5. **Monolith pages**: `app/chat/page.tsx` (~~4000+ lines) and~~ `app/crm/page.tsx` ~~(~~4600+ lines) are single-file monoliths with dozens of `useState` hooks. Difficult to maintain.
6. **Stream persistence hack**: The `_globalStream` module-level singleton + 80ms polling in `app/chat/page.tsx` is a workaround for Next.js navigation unmounting. Fragile.
7. **Duplicated token refresh**: `refreshIfNeeded()` for Google/Microsoft OAuth tokens is copy-pasted across ~8 route files. Divergence risk.
8. **No pagination on** `/api/email/messages`: Returns all rows — will not scale.
9. **FIFO trim on WhatsApp messages**: `saveEvolutionMessage` prunes to 30 messages per contact after each insert. Configurable via `EVOLUTION_FIFO_LIMIT` but defaults to 30.
10. `saveSlackConnection`: Uses select-then-insert instead of atomic upsert — race condition possible.

### 9.2 Functional Limitations

1. **No Google Meet creation route**: `/api/meet/oauth/google/connect` stores the OAuth token but no route ever creates a Meet space. The AI action `create_meet` creates a Calendar event with Meet conference data instead.
2. **Calendly upsert bug**: `/api/calendly/fetch` upsert checks `.eq("id", evt.id)` where `evt.id` is a Calendly URI suffix — likely mismatches the UUID `id` column, causing duplicate inserts.
3. `/api/chat/cleanup`: Takes `userId` but filters nothing by it — cleans the entire `chat_messages` table regardless of user.
4. **Email keyword filter**: `/api/email/fetch` (1245 lines) has a complex business-email filter (`BUSINESS_KEYWORDS`, `EXCLUDED_SENDERS`, `NEWSLETTER_INDICATORS`). Hardcoded, not configurable per user (though `profiles.email_keywords` exists for per-user keywords in CRM).

---

## 10. Outstanding Bugs, Errors, Unfinished Features & Technical Debt

### 10.1 Known Bugs

| # | Bug | Location |
|---|---|---|
| 1 | Calendly upsert uses URI suffix as `id` — causes duplicate event inserts | `/api/calendly/fetch` |
| 2 | `/api/chat/cleanup` ignores `userId` — cleans entire table | `app/api/chat/cleanup/route.ts` |
| 3 | `/api/whatsapp/embedded-signup` hardcoded redirect `https://private-gpt-psi.vercel.app/` | `app/api/whatsapp/embedded-signup/route.ts:41` |
| 4 | `/api/workspace/invitations` has no ownership/auth check | `app/api/workspace/invitations/route.ts` |
| 5 | `/api/messages/mark-read` has no userId check — can mark any message read by id | `app/api/messages/mark-read/route.ts` |

| | | |
|---|---|---|
| 6 | `/api/workspace/decline-invite` has no email match check — anyone with token can decline | `app/api/workspace/decline-invite/route.ts` |
| 7 | `saveSlackConnection` race condition (select-then-insert, not atomic upsert) | `lib/supabase.ts` |
| 8 | `/api/email/messages` returns all rows with no pagination | `app/api/email/messages/route.ts` |
| 9 | `/api/admin/lookup-user` does full `auth.admin.listUsers()` scan (no index) | `app/api/admin/lookup-user/route.ts` |
| 10 | `/api/slack/fetch` N+1 `users.info` calls — rate-limit prone | `app/api/slack/fetch/route.ts` |

### 10.2 Unfinished Features (Stub Pages)

| Route | Status |
|---|---|
| `/contacts` | "Coming Soon" placeholder (functionality in `/crm`) |
| `/inbox` | "Coming Soon" placeholder (functionality in `/crm` + `/chat`) |
| `/agents` | "Coming Soon" placeholder |
| `/analytics` | "Coming Soon" placeholder |
| `/automations` | "Coming Soon" placeholder |
| `/construction` | Fake email-capture (localStorage only, goes nowhere) |
| `/creating` | Cosmetic loading screen (hardcoded timers, no real processing) |

### 10.3 "Coming Soon" Channels

| Channel | Status |
|---|---|
| Website Chat | UI badge only |
| Microsoft Teams | UI badge only |
| Instagram DM | UI badge only |
| Facebook Messenger | UI badge only |
| SMS / Text Messages | UI badge only |

### 10.4 Roadmap Items Not Implemented

- Vector embeddings (pgvector + semantic search)
- Fully configurable AI agents with tool access and memory
- Trigger-based automation workflows (e.g., email received → AI draft reply)
- Analytics dashboard (conversation volume, response quality, token usage charts)
- CAC tracking (marketing spend → LTV/CAC reporting)
- WhatsApp Embedded Signup (guided WABA creation)
- Transify (translation workflow tool — page scaffolded, super-admin only)

### 10.5 Technical Debt

1. **Monolith pages**: `app/chat/page.tsx` and `app/crm/page.tsx` need to be broken into hooks/subcomponents.
2. **`lib/supabase.ts`** is ~2000 lines — should be split by domain (email, calendar, whatsapp, etc.).

3. **Duplicated** `refreshIfNeeded()` across ~8 route files — should be a shared utility.
4. **Excessive** `console.log` in production code (debug logging in supabase.ts, email/fetch, whatsapp sync, etc.).
5. `lib/translations.ts` is ~124KB — consider lazy-loading per-language.
6. **No test suite** beyond a single web-search integration test.
7. **No CI/CD pipeline** configured in the repo.
8. `tsconfig.tsbuildinfo` (1.1MB) is committed — should be in `.gitignore`.
9. `.DS_Store` files present in multiple directories.
10. **Lemon Squeezy columns** (legacy, replaced by Polar) still in DB — `024_add_lemonsqueezy_columns.sql`.

---

## 11. Credentials, Passwords & Access Transfer

The following credentials and access information must be transferred to Secretes Corp DBA Us+AI Bureau. **Do not include actual secret values in this document** — transfer them securely via a password manager or encrypted channel.

### 11.1 Application Secrets ( `.env` file)

All variables in `.env` must be transferred. The full list of required env vars is in `.env.example` (see §8 of this doc for the service mapping):

- Supabase (URL, anon key, service role key)
- DeepSeek (API key, URL, model)
- Serper (API key, URL)
- Resend (API key)
- Google OAuth (client ID, client secret)
- Microsoft OAuth (client ID, client secret)
- Meta (app ID, app secret, WhatsApp config ID, webhook verify token)
- Slack (client ID, client secret, bot token)
- Telegram (API ID, API hash)
- Calendly (client ID, client secret, webhook signing key)
- Stripe (publishable key, API key, webhook secret, price IDs for Solo + Team)
- Polar.sh (access token, webhook secret, product IDs for Solo + Team)
- Evolution API (URL, API key)
- Cron secret

### 11.2 Infrastructure Access

| Resource | Access Required |
|---|---|
| GitHub repo ( `PROYECT888/Private-GPT` ) | Admin/owner transfer |
| Vercel project | Project owner transfer |
| Supabase project | Project owner transfer |
| Hostinger VPS ( `srv1912272.hstgr.cloud` ) | Root SSH access + hPanel |
| Domain registrar (for `exploro-os.com` ) | Ownership transfer |
| Stripe dashboard | Account access |
| Polar.sh dashboard | Organization access |
| Google Cloud Console (OAuth app) | Project owner |

| | |
|---|---|
| Microsoft Azure (OAuth app) | App registration owner |
| Meta Developer (WhatsApp/IG/FB app) | App admin |
| Slack API (app config) | App owner |
| Telegram (API credentials) | App ownership |
| Calendly (OAuth app) | App owner |
| Resend (account) | Account access |
| DeepSeek (API account) | Account access |
| Serper (API account) | Account access |

### 11.3 VPS Access

- **Host**: `srv1912272.hstgr.cloud`
- **SSH**: `root@srv1912272.hstgr.cloud` (password from Hostinger hPanel)
- **Evolution API**: `/opt/evolution-api/.env` contains `AUTHENTICATION_API_KEY` and `DATABASE_URL` password
- **PostgreSQL**: `evolution` database, user `evolution`, password in `/opt/evolution-api/.env`
- **PM2**: `pm2 status` / `pm2 logs evolution-api`

### 11.4 Confirmation

I confirm that upon transfer of the above credentials, I will not retain, use, modify, disclose, copy, or otherwise access company credentials or project systems except where expressly authorized in writing.

---

## 12. Recurrent Problems & Troubleshooting History

### 12.1 PDF Extraction

See `docs/troubleshooting-pdf-extraction.md`. Key issues:

- `pdf-parse` fails on some PDFs (corrupted, scanned, encrypted).
- `pdfjs-dist` used as fallback for complex PDFs.
- Some PDFs return 0 pages or garbled text.

### 12.2 Supabase Email Storage

See `docs/troubleshooting-supabase-email-storage.md`. Key issues:

- Email body truncation for large emails.
- HTML body storage and rendering.
- Duplicate email prevention via unique constraints.

### 12.3 Department Heads & Web Search

See `docs/troubleshooting-dept-heads-and-web-search.md`. Key issues:

- Web search query rewrite accuracy.
- Domain authority reranking edge cases.

### 12.4 Google OAuth Verification

See `docs/google-oauth-verification-response.md` and `docs/google-scope-changes-and-casa-cost.md`. Key issues:

- Google CASA (Cloud Application Security Assessment) requirements for sensitive scopes.
- Scope changes and cost implications.
- OAuth verification response documentation.

### 12.5 WhatsApp Evolution API

Recurrent issues (from git log):

- Messages appearing then disappearing — race condition in realtime DELETE handler (fixed: merge instead of replace).
- Phone number `+` prefix stripping for Evolution API.
- Webhook event name casing + `@lid` phone extraction.
- Session disconnection/reconnection handling.
- Contact sync from VPS pushNames.
- FIFO trim causing message loss (raised to 500, then removed FIFO trim entirely for synced messages).

### 12.6 CRM WhatsApp Reply

- Reply target uses `to_number` for sent messages so thread replies work correctly.
- Send button on contact page opens WhatsApp compose modal.

### 12.7 Polar API Versioning

- Polar API pinned to `2026-04` version header ( `lib/polar.ts` ). Must update before each quarterly release.

### 12.8 Trial End Date Calculation

- Trial end date now extends from `now` when calculated end is in the past (was a bug where expired trials got negative days).

---

## 13. Development Decisions & Workarounds

1. **DeepSeek over OpenAI**: Chosen for cost efficiency. Model `deepseek-v4-flash` with prompt caching enabled. Cache hit ratio logged.
2. **Dual payment processors**: Stripe for US (seat-based Team billing), Polar.sh for non-US (Merchant of Record handles global tax/VAT). `CheckoutButton` routes based on `profile.location` .
3. **Evolution API over Meta Cloud API**: WhatsApp QR-based pairing (Evolution) is the primary provider ( `NEXT_PUBLIC_WHATSAPP_PROVIDER=evolution` ). Meta Cloud API is the fallback. Evolution allows personal WhatsApp accounts without a business WABA.
4. `teleproto` **over official Telegram SDK**: GramJS fork for MTProto user-account access (not bot API). Allows reading user DMs.
5. **Cookie-based auth storage**: Chosen so `middleware.ts` can read the session server-side for route protection. Trade-off: cookie is not HttpOnly (XSS risk).
6. `_globalStream` **singleton**: Workaround for Next.js App Router unmounting components during navigation. Keeps SSE stream alive at module level.
7. **Service-role key via** `NEXT_PUBLIC_` : Necessary because some client-side code (Evolution API functions in `lib/supabase.ts` ) needs admin access. **This is a security trade-off that should be fixed** by moving those functions to API routes.

8. **Keyword-based email filtering**: Hardcoded business keywords in `/api/email/fetch` to filter out newsletters/automated emails. Per-user keywords ( `profiles.email_keywords` ) added later for CRM.
9. **30-day document expiry**: Storage cost management. Pinned documents are exempt. `lazyArchiveDocuments` deletes storage files (not DB rows) for expired docs.
10. `sessionStorage` **CRM caching**: `crm_{userId}_{key}` pattern avoids flash of stale data on page reload.

---

## 14. Operational Runbook

### 14.1 Deploying Changes

```
# 1. Push to development branch
git push origin development

# 2. Vercel auto-deploys on push (preview deployment)
# 3. Promote to production via Vercel dashboard (or merge to main)
git checkout main
git merge development
git push origin main
# Vercel auto-deploys production
```

### 14.2 Running Database Migrations

```
# Apply new migration files in order from supabase/migrations/
# Via Supabase CLI:
supabase db push

# Or manually via Supabase Dashboard → SQL Editor
```

### 14.3 Restarting Evolution API (VPS)

```
ssh root@srv1912272.hstgr.cloud
pm2 restart evolution-api
pm2 logs evolution-api --lines 20
```

### 14.4 SSL Renewal (VPS)

```
ssh root@srv1912272.hstgr.cloud
certbot renew
```

```
systemctl reload nginx
```

### 14.5 Backup Evolution Database

```
ssh root@srv1912272.hstgr.cloud
pg_dump -U evolution -h localhost evolution > backup_$(date +%Y%m%d).sql
```

### 14.6 Updating Polar API Version

Update `POLAR_VERSION` in `lib/polar.ts` ( `polarHeaders()` ) before each quarterly release. See https://docs.polar.sh/api-versioning.

### 14.7 Adding a New Email Provider

1. Add to `EMAIL_PROVIDERS` array in `app/channels/page.tsx` (id, name, icon, SMTP/IMAP defaults, descKey).
2. Add translation keys to `lib/translations.ts` (EN + ES).
3. No backend changes needed — SMTP/IMAP is provider-agnostic.

### 14.8 Adding a New Messaging Channel

1. Add to `MESSAGING_CHANNELS` array in `app/channels/page.tsx` .
2. Set `connectable: true` if implemented, `false` for "Coming soon".
3. Add API routes under `app/api/{channel}/` .
4. Add DB table + migration if storing messages.
5. Add Supabase helpers to `lib/supabase.ts` .
6. Add realtime subscription helper.

---

## 15. File Reference Index

| File | Purpose |
|---|---|
| `README.md` | Project overview, setup, features, roadmap |
| `VPS_SETUP_GUIDE.md` | Evolution API VPS setup (Hostinger) |
| `.env.example` | All required env vars with descriptions |
| `middleware.ts` | Route protection (auth cookie check) |
| `next.config.mjs` | Image remote patterns, pdf-parse external package |
| `lib/supabase.ts` | All DB/Storage helpers (~2000 lines) |
| `lib/token-limits.ts` | Per-plan token/message caps + quota checking |
| `lib/subscription.ts` | Subscription helpers (getSubscription, startTrial, isTrialExpired) |
| `lib/polar.ts` | Polar.sh API client (pinned to 2026-04) |
| `lib/system-email.ts` | Transactional email templates (Resend) |

| | |
|---|---|
| `lib/telegram-user.ts` | Telegram MTProto user-session helpers |
| `app/auth-provider.tsx` | AuthContext (user, session, role, subscription) |
| `app/workspace-provider.tsx` | WorkspaceContext (multi-workspace) |
| `app/chat/page.tsx` | AI chat workspace (~4000+ lines) |
| `app/crm/page.tsx` | CRM with tabs (~4600+ lines) |
| `app/channels/page.tsx` | Integrations hub (~1700+ lines) |
| `app/admin/page.tsx` | Super admin panel (~2000+ lines) |
| `docs/troubleshooting-pdf-extraction.md` | PDF parsing issues |
| `docs/troubleshooting-supabase-email-storage.md` | Email storage issues |
| `docs/troubleshooting-dept-heads-and-web-search.md` | Web search issues |
| `docs/google-oauth-verification-response.md` | Google OAuth verification |
| `docs/google-scope-changes-and-casa-cost.md` | Google scope/CASA |
| `docs/CEO_Cost_Analysis_Per_User.md` | Per-user cost analysis |
| `docs/knowledge-base-spec.md` | Knowledge base spec |

---

## 16. Acknowledgment

I, Dhiren Kirpalani, acknowledge that this Technical Transition Letter documents the current architecture, codebase, database, APIs, integrations, infrastructure, known issues, and technical debt of Exploro OS as of October 12, 2026. I agree to complete the handover of all credentials, access information, and technical materials before October 15, 2026. I confirm that the confidentiality, intellectual-property, non-circumvention, and other applicable obligations contained in the LoI continue to apply after termination where stated therein.

---

**End of Technical Transition Letter — Exploro OS**